1. Map-Coloring Problem

```
# Map coloring using Backtracking and Constraint Satisfaction.

graph = {
    "A": ["B", "C", "D"],
    "B": ["A", "C", "E"],
    "C": ["A", "B", "D", "E"],
    "D": ["A", "C", "E"],
    "E": ["B", "C", "D"]
}

colors = ["Red", "Green", "Blue"]
assignment = {}

def is_valid(region, color):
    for neighbor in graph[region]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve():
    if len(assignment) == len(graph):
        return True

    # Select the next unassigned region.
    region = next(r for r in graph if r not in assignment)

    for color in colors:
        if is_valid(region, color):
            assignment[region] = color

            if solve():
                return True

            del assignment[region]

    return False

if solve():
    print("Map coloring solution:")
    for region, color in assignment.items():
        print(region, ":", color)
else:
    print("No solution exists.")
```

2. Crypt-Arithmetic Puzzle

```
# Solve the SEND + MORE = MONEY puzzle.
# Each letter represents a different digit.

letters = "SENDMORY"
solution = {}

def solve_cryptarithm():
    digits = set(range(10))

    def search(index, used):
        if index == len(letters):
            send = (solution["S"] * 1000 +
                    solution["E"] * 100 +
                    solution["N"] * 10 +
                    solution["D"])
            more = (solution["M"] * 1000 +
                    solution["O"] * 100 +
                    solution["R"] * 10 +
                    solution["E"])
            money = (solution["M"] * 10000 +
                     solution["O"] * 1000 +
                     solution["N"] * 100 +
                     solution["E"] * 10 +
                     solution["Y"])
            return send + more == money

        letter = letters[index]

        for digit in digits - used:
            if letter in ("S", "M") and digit == 0:
                continue

            solution[letter] = digit

            if search(index + 1, used | {digit}):
                return True
```

```
            del solution[letter]

        return False

    return search(0, set())

if solve_cryptarithm():
    print("Solution:")
    for letter in letters:
        print(letter, "=", solution[letter])

    send = (solution["S"] * 1000 + solution["E"] * 100 +
            solution["N"] * 10 + solution["D"])
    more = (solution["M"] * 1000 + solution["O"] * 100 +
            solution["R"] * 10 + solution["E"])
    money = (solution["M"] * 10000 + solution["O"] * 1000 +
             solution["N"] * 100 + solution["E"] * 10 +
             solution["Y"])

    print(send, "+", more, "=", money)
else:
    print("No solution exists.")
```

3. Local Search for CSP (N-Queens using Min-Conflicts)

```
# Solve the N-Queens CSP using the Min-Conflicts algorithm.
# Each column contains exactly one queen.
# A state stores the row position of each column's queen.

import random

def conflicts(state, column, row):
    count = 0

    for other_col in range(len(state)):
        if other_col == column:
            continue

        other_row = state[other_col]

        if other_row == row:
            count += 1
        if abs(other_row - row) == abs(other_col - column):
            count += 1

    return count

def min_conflicts(n, max_steps=100000):
    state = [random.randrange(n) for _ in range(n)]

    for _ in range(max_steps):
        conflicted = [
            col for col in range(n)
            if conflicts(state, col, state[col]) > 0
        ]

        if not conflicted:
            return state

        column = random.choice(conflicted)

        scores = [
            conflicts(state, column, row)
            for row in range(n)
        ]

        minimum = min(scores)
        best_rows = [
            row for row in range(n)
            if scores[row] == minimum
        ]

        state[column] = random.choice(best_rows)

    return None

n = int(input("Enter number of queens: "))

if n < 4:
    print("Enter N greater than or equal to 4.")
else:
    solution = min_conflicts(n)

    if solution is None:
        print("Solution not found within the step limit.")
```

```
else:
    print("Solution (row positions by column):", solution)

    for row in range(n):
        print(" ".join(
            "Q" if solution[col] == row else "."
            for col in range(n)
        ))
```